DWES - Spring Boot - Procedimiento de inicio del examen

Guía offline desde Spring Initializr hasta tener la base lista para programar

Pensada para un examen sin Internet, con entrega final en ZIP por Moodle

0. Regla principal

SIEMPRE	Lee primero el enunciado completo y la rúbrica. No añadas cosas solo porque aparecen en Biblioteca Decroly si el examen no las pide.
SIEMPRE	Una vez cortado Internet, trabaja con archivos locales: proyectos de referencia, PDFs, este manual y el catálogo HTML de endpoints.
NO DEPENDER	No bases el examen en GitHub, el workspace cloud de Postman, Maven Wrapper ni páginas web. Pueden necesitar conexión.

Objetivo práctico: tener una API arrancando cuanto antes y construir después solo lo que exige el enunciado.

1. Antes de que corten Internet

1. Enciende el PC y abre únicamente lo imprescindible.
2. Descarga el proyecto desde Spring Initializr mientras todavía haya Internet.
3. Comprueba que tienes disponibles localmente tus PDFs y proyectos de referencia. No confíes en enlaces online.
4. Ten localizada la carpeta donde guardarás el proyecto del examen.
5. Cuando el profesor indique que se corta Internet, no dependas de sincronizaciones o descargas posteriores.

2. Spring Initializr: configuración base

Campo	Valor recomendado	Por qué
Project	Maven	Es el sistema de construcción usado en clase.
Language	Java	Lenguaje trabajado.
Spring Boot	4.1.1 si está disponible y coincide con el entorno de clase	Es la versión usada por Biblioteca Decroly y ya se ha utilizado en clase.
Group	El que indique el examen; si no, es.decroly	Define la raíz del paquete.

Artifact	Nombre del proyecto en minúsculas y con guiones	Ej.: gestion-cursos.
Packaging	Jar	Configuración habitual trabajada.
Java	21	Versión del proyecto de Biblioteca Decroly.

SIEMPRE	Añade Spring Web. Es la dependencia base para crear controladores REST y levantar Tomcat.
SIEMPRE	Añade Lombok si vas a usar @Getter, @Setter, @RequiredArgsConstructor, etc. Es lo que has usado en Biblioteca.
EVITAR	No añadas Spring Boot DevTools salvo que el profesor lo pida. No aporta nada esencial al examen y puede requerir una descarga que no esté en caché.
SI LO PIDE	Si el examen exige Swagger/OpenAPI, añade después springdoc 3.2.0 en pom.xml. Si no pide documentación, omítelo.

```
<!-- Solo si el enunciado pide OpenAPI / Swagger -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>3.2.0</version>
</dependency>
```

3. En cuanto descargues el ZIP

6. Guarda el ZIP en una carpeta fácil de localizar.
7. Extrae el ZIP. No trabajes dentro del archivo comprimido.
8. Abre IntelliJ IDEA.
9. Pulsa Open y selecciona la carpeta extraída que contiene pom.xml.
10. Acepta Trust Project si IntelliJ lo pregunta.
11. Espera a que IntelliJ importe Maven. Si todo está cacheado en el PC de clase, no debería necesitar Internet.
12. Ve a File > Project Structure > Project y comprueba SDK Java 21 y Language level SDK default.
13. Ejecuta la clase principal *Application antes de programar. Si arranca, ya tienes una base sana.

```
Tomcat started on port 8080
Started ...Application
```

4. Estructura de paquetes: qué crear y cuándo

Elemento	¿Cuándo?	Contenido
model	SIEMPRE si hay entidades	Clases principales: Libro, Categoria, Producto, Alumno...
service	SIEMPRE en este enfoque por capas	Interfaz Service + implementación con @Service.
controller	SIEMPRE para la API REST	Rutas, parámetros HTTP y delegación al Service.
exception	SIEMPRE si hay errores propios	400, 404, 409 y manejador global con ProblemDetail.
dto	SI LO PIDE / cuando conviene	Datos de entrada distintos de la entidad o respuestas calculadas.
config	SI LO PIDE	@ConfigurationProperties, configuración propia, OpenAPI, etc.
resources/application.properties	SIEMPRE existe	Nombre de aplicación y propiedades que pida el examen.

SIEMPRE

Crea todos estos paquetes debajo del paquete raíz generado por Spring Initializr. Si los sacas fuera, Spring puede no encontrarlos mediante el escaneo de componentes.

5. Orden universal para desmontar el enunciado

14. Subraya las ENTIDADES. Ej.: Libro y Categoria.
15. Apunta todos los CAMPOS de cada entidad y sus tipos.
16. Marca reglas: obligatorio, único, mínimo, máximo, relación, estado, etc.
17. Haz una tabla rápida de ENDPOINTS: método, ruta, entrada, salida y errores.
18. Decide si necesitas DTO para creación/modificación o para respuestas calculadas.
19. Decide las excepciones necesarias: datos inválidos, no encontrado, conflicto.
20. Crea modelos y DTO.
21. Crea las interfaces Service.
22. Implementa ServiceImpl con listas en memoria y toda la lógica de negocio.
23. Crea Controllers que solo reciban, deleguen y devuelvan la respuesta HTTP.
24. Crea o adapta el manejador global de errores.
25. Añade configuración propia solo si se pide.
26. Prueba endpoint por endpoint antes de pasar al siguiente bloque.

6. Qué es realmente común a casi cualquier caso

Patrón	Dónde mirarlo en Biblioteca Examen
CRUD GET/POST/PUT/DELETE	CategoriaControlador, LibroControlador y sus Services
Buscar por ID + 404	obtenerPorId(...)
Validar datos + 400	validarLibro(...) / DatosNoValidosException

Campo único + 409	ISBN y nombre de categoría
Relación entre entidades	categoriaId y listarPorCategoria(...)
Filtros	LibroServicioImpl.filtrar(...)
Resultado calculado	CategoriaResumenDto + BibliotecaGestionServicio
Regla antes de borrar	eliminarCategoriaSiEstaVacia(...)
Propiedades	BibliotecaPropiedades + application.properties
ProblemDetail	ManejadorGlobalExcepciones

7. Trabajar y consultar ejemplos sin Internet

SIEMPRE	Lleva una copia local de biblioteca-decroly-examen-kit-offline. Dentro tendrás código, GUIA_EXAMEN.md y GUIA_ENDPOINTS_OFFLINE.html.

Para localizar código rápido en IntelliJ: Ctrl + Shift + F y busca EJEMPLO EXAMEN o palabras como FILTRO POR RANGO, CAMPO ÚNICO, REGLA AL BORRAR, RESULTADO CALCULADO.

Para ver todos los endpoints sin Internet: abre GUIA_ENDPOINTS_OFFLINE.html con Chrome/Edge y usa Ctrl + F. No necesita servidor ni conexión.

Para probar una API local sin depender de Postman: usa el navegador para GET sencillos o curl.exe desde PowerShell/CMD.

```
curl.exe http://localhost:8080/api/libros
curl.exe -X DELETE http://localhost:8080/api/libros/1
```

8. Postman durante un examen sin Internet

PRECAUCIÓN	No des por hecho que un workspace sincronizado con tu cuenta cargará sin Internet. En la prueba de casa, el workspace cloud se quedó intentando cargar.
PLAN A	Si Postman en el PC de clase funciona offline, úsalo normalmente.
PLAN B	Si no carga, prueba la API con curl.exe y usa la colección JSON solo como referencia/entregable si el enunciado la exige.
SI LO PIDE	Si la colección Postman forma parte de la entrega, habrá que seguir un procedimiento específico de creación/exportación local. No dependas de una colección que solo exista en la nube.

9. Swagger/OpenAPI: solo si aparece en el enunciado

- Si NO aparece OpenAPI, Swagger, @Tag, @Operation, @ApiResponse o documentación: no pierdas tiempo añadiéndolo.
- Si SÍ aparece: añade springdoc, documenta los endpoints y usa /swagger-ui.html y /v3/api-docs.
- La API REST funciona perfectamente sin Swagger; Swagger es documentación, no la API en sí.

10. Checklist de los primeros 15 minutos

	Comprobación
[ ]	He leído TODO el enunciado y la rúbrica.
[ ]	He identificado entidades, campos y reglas.
[ ]	He descargado Initializr antes del corte de Internet.
[ ]	Proyecto extraído y abierto por la carpeta que contiene pom.xml.
[ ]	JDK 21 configurado.
[ ]	La aplicación arranca antes de empezar.
[ ]	He creado solo los paquetes que voy a necesitar.
[ ]	Tengo abierta la guía offline y Biblioteca Examen como referencia.
[ ]	He decidido qué endpoint voy a implementar primero.

11. Cosas que NO hacer por inercia

- No copies Biblioteca entera y hagas reemplazo masivo de nombres sin leer el enunciado.
- No pongas lógica de negocio en el Controller.
- No uses @Autowired en atributos si puedes usar constructor / @RequiredArgsConstructor.
- No añadas dependencias que no necesitas.
- No dependas de GitHub durante el examen.
- No ejecutes Maven Wrapper como paso obligatorio si el equipo intenta descargar Maven.
- No añadas Swagger si no lo piden.
- No olvides que un filtro sin coincidencias normalmente devuelve 200 + lista vacía; un recurso concreto inexistente devuelve 404.

12. Fuentes de clase usadas para esta guía

UD3 3.1 Primer proyecto Spring Boot; UD3 3.2 API REST - Capa Controller; UD3 3.3 La capa Service y la configuración; UD3 3.4 Consumir y documentar APIs; proyecto Biblioteca Decroly / Biblioteca Decroly Examen.